

一、先搞懂：CSP到底是干嘛的？

CSP (Content Security Policy , 内容安全策略) , 是浏览器层面为了防御XSS (跨站脚本攻击) 而生的终极安全机制。

你可以把它理解成网站给浏览器装的一道「安全防盗门」：

正常的CSP，会给浏览器列一份「白名单」：只允许加载/执行白名单里的可信资源（比如自己服务器的JS、可信CDN的样式）。

不在白名单里的资源、内联脚本、恶意代码，浏览器会直接拦截，从根源上阻断XSS攻击。

而我们遇到的所有问题，本质都是：亲手给这道防盗门开了无数个大洞，让CSP彻底形同虚设。

二、CSP高危漏洞全拆解

1. 漏洞1：CSP中允许内联脚本执行（高危）

漏洞名称

CSP中允许内联脚本执行

漏洞原因

在`script-src`指令中配置了`'unsafe-inline'`，强制浏览器放行所有内联脚本（包括直接写在HTML里的`<script>`代码、`onclick`等内联事件）。

正常CSP默认禁止所有内联脚本，只有手动添加`'unsafe-inline'`才会放行。

这个配置等于告诉浏览器：“不管是我自己写的正常JS，还是黑客注入的恶意内联脚本，全给我放行”，直接废掉了CSP的核心防护。

风险后果

黑客只要能在网站任何输入点（评论区、个人资料、URL参数）注入一段恶意内联脚本，就能直接在用户浏览器执行，偷取Cookie、盗号、弹钓鱼广告，XSS攻击零门槛。

解决方案

将所有内联脚本、内联事件提取到独立的`.js`外部文件，彻底删除`'unsafe-inline'`。

必须保留内联脚本的场景：使用`nonce`（随机数）或`hash`（脚本哈希）白名单，仅放行可信内联脚本，不全局开放。

2. 漏洞2：在CSP中的不安全URL方案（高危）

漏洞名称

CSP中使用不安全URL方案 (data:/blob:)

漏洞原因

在script-src等指令中配置了data:和blob:协议，允许浏览器加载/执行用这两个协议打包的资源。

data:/blob:是特殊的打包协议，可以把JS、图片等资源直接编码嵌在URL里，不用单独存成文件。

开放data:/blob:给脚本，等于给黑客开了“把恶意代码藏在特殊链接里进门”的大门，CSP完全无法拦截。

风险后果

黑客可以把恶意JS转成base64，用data:text/javascript;base64,xxx的形式注入，浏览器直接执行，结合'unsafe-inline'形成双重漏洞叠加，XSS攻击成功率拉满。

解决方案：

直接从script-src中删除data:和blob:，仅给图片 (img-src) 等非脚本资源按需放行。

必须使用的场景：严格限制仅放行特定类型（如仅图片），绝对不允许脚本类型的data:/blob:。

3. 漏洞3：在源指令中过度使用宽泛的通配符（高危）

漏洞名称

CSP源指令过度使用*通配符

漏洞原因

在script-src、base-uri、form-action等指令中使用了*通配符，代表“允许任意来源的所有资源”。

正常CSP应该只放行可信域名（如'self'、可信CDN域名），*通配符等于完全放弃白名单能力。

这个配置等于告诉浏览器：“不管脚本来自哪个网站，是合法的还是黑客的，全给我放行”。

风险后果

黑客可以从任何恶意网站、广告链接引入恶意JS，网站毫无阻拦地执行，彻底失去来源管控能力，XSS、数据泄露风险极高。

解决方案

彻底删除所有*通配符，替换为具体可信域名，如script-src 'self' https://trusted.cdn.com。

核心原则：脚本 (script-src) 绝对禁止使用*通配符，仅在必要的非脚本资源中谨慎使用。

4. 漏洞4：绕过对象白名单配置（高危）

漏洞名称漏洞名称

CSP对象源白名单绕过 (`object-src`全放行)

漏洞原因

`object-src`指令配置为`* data: blob:`，无限制放行所有插件/嵌入式对象。

`object-src`是CSP中专门管控Flash、`<object>/<embed>`等插件资源的指令，现代网站几乎不再使用这类插件。

全放行配置等于完全无限制插件来源，黑客可以注入恶意Flash/嵌入式对象，直接在用户浏览器执行。

风险后果

黑客可以通过恶意插件执行任意代码，绕过CSP的脚本防护，实现提权、数据窃取等攻击，属于额外的攻击入口。

解决方案

最优方案：直接配置`object-src 'none'`，完全禁用所有插件/嵌入式对象，现代网站99%场景适用，对业务零影响。

必须使用插件的场景：仅放行可信域名，绝对禁止`*`、`data:`、`blob:`。

5. 漏洞5：在CSP中使用了不安全的eval()（中危）

漏洞名称

CSP中允许`'unsafe-eval'`

漏洞原因

在`script-src`指令中配置了`'unsafe-eval'`，允许浏览器执行`eval()`、`new Function()`等动态代码。

正常CSP默认禁止`eval()`执行，手动添加`'unsafe-eval'`才会放行。

这个配置等于给黑客开了“动态代码执行”的绿灯，黑客可以用`eval()`解密、变形恶意代码，绕过WAF和前端防护。

风险后果

放大XSS攻击威力，让黑客可以用更隐蔽的方式注入恶意代码，即使有基础防护也能轻易绕过。

解决方案

直接删除`'unsafe-eval'`，重构代码，用静态代码替代`eval()`等动态执行逻辑（如用`JSON.parse()`替代`eval()`解析JSON）。

必须使用的场景：严格限制使用范围，不全局开放`'unsafe-eval'`。

6. 低危补充：在CSP中未强制执行脚本中的可信类型

漏洞名称

CSP未开启可信类型（Trusted Types）

漏洞原因

未配置`require-trusted-types-for 'script'`指令，未开启CSP的进阶DOM XSS防护。

可信类型是CSP的扩展防护，强制浏览器只允许执行经过验证的可信脚本，拦截危险的DOM操作（如`innerHTML`）。

未配置该指令，等于没装“第二道锁”，DOM XSS风险依然存在。

风险后果

属于锦上添花的防护缺失，优先级低于上述5个高危漏洞，不影响核心安全，但会降低XSS防护能力。

解决方案

在CSP中添加`require-trusted-types-for 'script'; trusted-types default;`，开启可信类型防护。

前端无法适配的场景：可标记为低危不整改，不影响等保/密评合规。

三、所有CSP漏洞的本质总结

很多人会觉得这是5个独立的问题，但实际上，它们的本质完全一致：

我们配置的CSP，不是在“防护”，而是在主动告诉浏览器：

不管什么JS、谁发来的、哪里来的、用什么方式包装的，全都允许执行。

等于给网站装了防盗门，却亲手把锁拆了、门大开，还贴了纸条：“黑客请随意进”。

这5个配置同时存在，CSP的XSS防护等于彻底报废，没有任何防御能力，黑客可以从任何地方、用任何方法把恶意代码塞进来，浏览器全执行，没有任何阻拦。

四、最终修复：一份可直接复制的安全CSP模板

把所有高危配置一次性替换，给你一份零风险的标准CSP配置：

```
Content-Security-Policy:
# 默认只允许自家域名的资源
default-src 'self';
# 脚本只允许自家服务器，彻底删除unsafe-inline、unsafe-eval、data:、blob:、*
script-src 'self';
# 样式允许自家和inline（样式inline通常可保留，脚本inline必须删）
style-src 'self' 'unsafe-inline';
# 图片允许自家和data:（仅图片，脚本绝对不加）
img-src 'self' data;;
```

```
# 字体只允许自家
font-src 'self';
# 接口请求只允许自家
connect-src 'self';
# 媒体资源只允许自家
media-src 'self';
# 直接禁用所有插件，彻底堵死object-src漏洞
object-src 'none';
# 禁止篡改页面基础URL
base-uri 'self';
# 表单只能提交到自家域名
form-action 'self';
# iframe只允许自家
frame-src 'self';
# Worker脚本只允许自家
worker-src 'self';
# 子资源只允许自家
child-src 'self';
# 开启可信类型进阶防护（可选，前端适配后添加）
# require-trusted-types-for 'script'; trusted-types default;
```

核心配置原则

1. 脚本 (script-src) 绝对红线：禁止'unsafe-inline'、'unsafe-eval'、data:、blob:、*通配符，只放行可信域名。
2. 非脚本资源按需放行：仅给图片、字体等非脚本资源，按需放行data:，绝不给脚本开放。
3. 插件彻底禁用：object-src 'none'，现代网站完全用不到，零业务影响，彻底堵死漏洞。
4. 通配符零容忍：所有指令彻底删除*，只配置具体可信域名。

五、修复后验证

1. 浏览器控制台验证：打开F12控制台，查看是否有CSP相关报错，确保业务功能正常。
2. 安全扫描验证：重新执行安全扫描，确认所有CSP相关告警已消除。
3. CSP校验工具验证：使用CSP Evaluator等工具，验证CSP策略的安全性。
4. 等保/密评合规验证：确认配置符合等保2.0、密评的安全要求，无高危配置。

六、写在最后

CSP是防御XSS的最后一道防线，而我们遇到的这些问题，本质都是“主动放弃了这道防线”。

很多时候，开发为了兼容业务、快速解决CSP拦截报错，会随手加上'unsafe-inline'、*、data:这些“万能钥匙”，却忽略了这些配置带来的致命安全风险。

CSP的核心是「限制」，不是「放行」。